SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO4: *Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25
Duration : 1 Hour

Industry Problem

Code of Conduct:

I A.Manoj Reddy 192311171 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING
ASSESSMENT TOOLS

1.Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module sh ould accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$L = \{w \mid w \in (0+1)* \; and \; n_0(w) = n_1(w)\}$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

L={a^n b^2n | n≥1}

4. Develop a **C-based computational module** to determine the **ε-closure of all states** in a **Non-Deterministic Finite Automaton (NFA) with ε-transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- **S → 0 A 1**
- **A → 0 A | 1 A | ε**

C program to simulate a DFA for strings that begin with a and end with a.

DFA states:-

- q_0 = initial state
- q_1 = string started with a and currently ends with a
- q_2 = string stared with a and currently ends with another symbol
- q_d = dead state

Assuming the alphabet is {a,b}:

Present State	Input a	Input b
q_0	q_1	q_d
q_1	q_1	q_2
q_2	q_1	q_2
q_d	q_1	q_d

C program:-

```
#include <studio.h>
#include <string.h>
int main()
{
    char Str[100];
    int state = 0;
    Printf ("Enter the String:");
    Scanf ("%s", Str);
```

```
for (int i=0; i< strlen(str); i++)
{
    switch (state)
    {
     case 0:
       if (str[i] == 'a')
            state = 1;
        else
            State = 3;
          break;
      Case 1:
        if (str[i] == 'a')
            State = 1;
        else if (str[i] == 'b')
            State = 2;
        else
            State = 3;
          break;
    Case 2:
          if (str[i] == 'a')
              State = 1;
          else if (str[i] == 'b')
              state = 2;
          else
              state = 3;
            break;
    }
    }
```

```
if (state ==1)
    Printf ("Accepted \n");
else:
    Printf ("Rejected\n");
    return 0;
}
```

Input: abba

Output: Accepted

Input: abb

output: Rejected.

PDA for equal number of 0's and 1's

The language given in the question is

$$L = \{ w \mid w \in (0+1)^* \text{ and } n_0(w) = n_1(w) \}$$

Idea:-

Use the stack to keep track of the unmatched symbols

- if 0 is read and stack top contains 1, POP it
- otherwise push 0.
- if 1 is read and the stack top contains 0, POP it.
- other wise push 1
- Accept when the complete input is processed and only the bottom stack symbol remains.

let:

$$M = (\{q_0, q_f\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \{q_f\})$$

transitions.

$$\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 1) = (q_0, 11)$$

when opposite symbols occur:

$$\delta(q_0, 01) = (q_0, \epsilon)$$

$$\delta(q_0, 1, 0) = (q_0, \epsilon)$$

finally $\delta(q_0, \epsilon, Z_0) = (q_f, Z_0)$

Example:-

for $\omega = 0101$

Stack processing!

Input	Stack
0	0
1	ϵ
0	0
1	ϵ

$n_0(\omega) = 2$, $n_1(\omega) = 2$

Hence 0101 is accepted.

PDA for $L = \{ a^n b^{2n} \mid n \geq 1 \}$

examples:

abb

aabbbb

aaabbbbbb

PDA Idea

for every a, push two stack symbols X

for every b, pop one X.

Therefore:

- 1 a → 2 X's
- 2 as → 4 X's
- 3 as → 6 X's

Accept when all input symbols are consumed and all X's are removed

$M = (\{q_0, q_1, q_f\}, \{a, b\}, \{X, Z_0\}, \delta, q_0, Z_0, \{q_f\})$

transitions:-

for first a:

$\delta(q_0, a, Z_0) = (q_0, XZ_0)$

for every additional a:

$\delta(q_0, a, X) = (q_0, XX)$

this effectively adds two new X's.

4.

ε - Closure of NFA

C program:-

```
# include <stdio.h>
int e[10][10], v[10], n;
void closure (int s){
    v[s]=1
for (int i=0; i<n; i++)
{
if (e[s][i] && !v[i])
        closure(i);
}
int main(){
    int m, a, b;
    Scanf ("%d %d", &n, &m);
while (m--){
    Scanf ("%d %d", &a, &b);
    e[a][b] = 1;
}
for (int s=0; s<n; s++){
    for (int i=0; i<n; i++) v[i]=0;
    closure(s);
    Printf ("e-closure(%d):", s);
    for (int i=0; i<n; i++)
```

```
        if (v[i]) printf("%d ", i);
        printf("\n");
    }
    return 0;
}
```

CFG validation:-

$S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \epsilon$

```
#include <stdio.h>
#include <string.h>
int main(){
    char s[100];
    scanf("%s", s);
    int n = strlen(s);
    if (n>=2 && s[0]=='0' && s[n-1]=='1')
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

Input: 00101 → Accepted

1010 → Rejected.

SIMATS ENGINEERING
ASSESSMENT TOOLS

:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

86/100